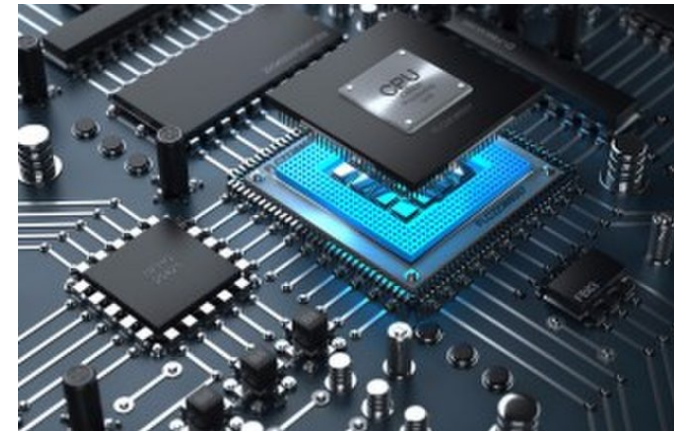


```
loop: lw    $t3, 0($t0)
      lw    $t4, 4($t0)
      add   $t2, $t3, $t4
      sw    $t2, 8($t0)
      addi  $t0, $t0, 4
      addi  $t1, $t1, -1
      bgtz  $t1, loop
```

Assembler

```
0x8d0b0000
0x8d0c0004
0x016c5020
0xad0a0008
0x21080004
0x2129ffff
0x1d20fff9
```



# Lecture 06:

# Assembly Programming Foundations

## *Branching Instructions*

---

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

Legend: Adm. Grace Hopper coined the term "debugging" when a moth was removed from the computer she was working on (see below – an actual page from her notebook from the 1950s)



# This Week on “Didja Know Dat?!”

0800 Antan started  
1000 " stopped - antan ✓  
1300 (032) MP-MC ~~1.582147000~~ 2.130476415 (23) 4.615925059(-2)  
(033) PRO 2 2.130476415  
conch 2.130676415  
Relays 6-2 in 033 failed special speed test  
in relay " 11.000 test.  
Relays changed  
1100 Started Cosine Tape (Sine check)  
1525 Started Multi-Adder Test.  
1545  Relay #70 Panel F  
(moth) in relay.  
First actual case of bug being found.  
1630 Antan started.  
1700 closed down.

Relay 2145  
Relay 3376



Reality: The term “bug” was used in engineering as early as the 1800’s.  
As seen independently from various scientists,  
including Ada Lovelace and Thomas Edison.

# Administrative

---

Assignment #3 is out and due *Monday!*

Remember: Lab on Friday!!

*A worksheet file is available for today's lecture on Canvas!*

*Pull up the MIPS Reference Sheet PDF for today*

# Bring out your MIPS Reference Cards!

*See front page of Canvas – link is at the bottom*

*(also found in this week's Canvas Module)*

| CORE INSTRUCTION SET        |       | FOR- | OPERATION (in Verilog)                                                       | OPCODE / FUNCT (Hex)      |
|-----------------------------|-------|------|------------------------------------------------------------------------------|---------------------------|
| NAME, MNEMONIC              | MAT   |      |                                                                              |                           |
| Add                         | add   | R    | $R[rd] = R[rs] + R[rt]$                                                      | (1) 0 / 20 <sub>hex</sub> |
| Add Immediate               | addi  | I    | $R[rt] = R[rs] + \text{SignExtImm}$                                          | (1,2) 8 <sub>hex</sub>    |
| Add Imm. Unsigned           | addiu | I    | $R[rt] = R[rs] + \text{SignExtImm}$                                          | (2) 9 <sub>hex</sub>      |
| Add Unsigned                | addu  | R    | $R[rd] = R[rs] + R[rt]$                                                      | 0 / 21 <sub>hex</sub>     |
| And                         | and   | R    | $R[rd] = R[rs] \& R[rt]$                                                     | 0 / 24 <sub>hex</sub>     |
| And Immediate               | andi  | I    | $R[rt] = R[rs] \& \text{ZeroExtImm}$                                         | (3) c <sub>hex</sub>      |
| Branch On Equal             | beq   | I    | if( $R[rs] == R[rt]$ )<br>$PC = PC + 4 + \text{BranchAddr}$                  | (4) 4 <sub>hex</sub>      |
| Branch On Not Equal         | bne   | I    | if( $R[rs] != R[rt]$ )<br>$PC = PC + 4 + \text{BranchAddr}$                  | (4) 5 <sub>hex</sub>      |
| Jump                        | j     | J    | $PC = \text{JumpAddr}$                                                       | (5) 2 <sub>hex</sub>      |
| Jump And Link               | jalu  | J    | $R[31] = PC + 8; PC = \text{JumpAddr}$                                       | (5) 3 <sub>hex</sub>      |
| Jump Register               | jr    | R    | $PC = R[rs]$                                                                 | 0 / 08 <sub>hex</sub>     |
| Load Byte Unsigned          | lbu   | I    | $R[rt] = \{24'b0, M[R[rs] + \text{SignExtImm}](7:0)\}$                       | (2) 24 <sub>hex</sub>     |
| Load Halfword Unsigned      | lhu   | I    | $R[rt] = \{16'b0, M[R[rs] + \text{SignExtImm}](15:0)\}$                      | (2) 25 <sub>hex</sub>     |
| Load Linked                 | ll    | I    | $R[rt] = M[R[rs] + \text{SignExtImm}]$                                       | (2,7) 30 <sub>hex</sub>   |
| Load Upper Imm.             | lui   | I    | $R[rt] = \{\text{imm}, 16'b0\}$                                              | f <sub>hex</sub>          |
| Load Word                   | lw    | I    | $R[rt] = M[R[rs] + \text{SignExtImm}]$                                       | (2) 23 <sub>hex</sub>     |
| Nor                         | nor   | R    | $R[rd] = \sim (R[rs]   R[rt])$                                               | 0 / 27 <sub>hex</sub>     |
| Or                          | or    | R    | $R[rd] = R[rs]   R[rt]$                                                      | 0 / 25 <sub>hex</sub>     |
| Or Immediate                | ori   | I    | $R[rt] = R[rs]   \text{ZeroExtImm}$                                          | (3) d <sub>hex</sub>      |
| Set Less Than               | slt   | R    | $R[rd] = (R[rs] < R[rt]) ? 1 : 0$                                            | 0 / 2a <sub>hex</sub>     |
| Set Less Than Imm.          | slti  | I    | $R[rt] = (R[rs] < \text{SignExtImm}) ? 1 : 0$                                | (2) a <sub>hex</sub>      |
| Set Less Than Imm. Unsigned | sltiu | I    | $R[rt] = (R[rs] < \text{SignExtImm}) ? 1 : 0$                                | (2,6) b <sub>hex</sub>    |
| Set Less Than Unsig.        | sltu  | R    | $R[rd] = (R[rs] < R[rt]) ? 1 : 0$                                            | (6) 0 / 2b <sub>hex</sub> |
| Shift Left Logical          | sll   | R    | $R[rd] = R[rt] \ll \text{shamt}$                                             | 0 / 00 <sub>hex</sub>     |
| Shift Right Logical         | srl   | R    | $R[rd] = R[rt] \gg \text{shamt}$                                             | 0 / 02 <sub>hex</sub>     |
| Store Byte                  | sb    | I    | $M[R[rs] + \text{SignExtImm}](7:0) = R[rt](7:0)$                             | (2) 28 <sub>hex</sub>     |
| Store Conditional           | sc    | I    | $M[R[rs] + \text{SignExtImm}] = R[rt];$<br>$R[rt] = (\text{atomic}) ? 1 : 0$ | (2,7) 38 <sub>hex</sub>   |
| Store Halfword              | sh    | I    | $M[R[rs] + \text{SignExtImm}](15:0) = R[rt](15:0)$                           | (2) 29 <sub>hex</sub>     |
| Store Word                  | sw    | I    | $M[R[rs] + \text{SignExtImm}] = R[rt]$                                       | (2) 2b <sub>hex</sub>     |
| Subtract                    | sub   | R    | $R[rd] = R[rs] - R[rt]$                                                      | (1) 0 / 22 <sub>hex</sub> |
| Subtract Unsigned           | subu  | R    | $R[rd] = R[rs] - R[rt]$                                                      | 0 / 23 <sub>hex</sub>     |

## NOTE THE FOLLOWING on the FRONT PAGE:

1. **Format Types** of each instruction: R vs I vs J

2. **OPCODE/FUNCT (Hex)**

These are unique numerical codes to distinguish each instruction from the other

3. **Operation Details** of each instruction  
e.g., for **add**, it's  $R[rd] = R[rs] + R[rt]$

4. **Instruction formats:**

Where the actual bits go, *i.e.*, how any assembly instruction gets translated into machine language  
(more on that in a [later lecture](#))

BASIC INSTRUCTION FORMATS

| R | opcode | rs      | rt    | rd        | shamt | funct |
|---|--------|---------|-------|-----------|-------|-------|
|   | 31     | 26 25   | 21 20 | 16 15     | 11 10 | 6 5   |
|   | 0      |         |       |           |       |       |
| I | opcode | rs      | rt    | immediate |       |       |
|   | 31     | 26 25   | 21 20 | 16 15     |       | 0     |
| J | opcode | address |       |           |       |       |
|   | 31     | 26 25   |       |           |       | 0     |

**PSEUDOINSTRUCTION SET**

| NAME                         | MNEMONIC | OPERATION                   |
|------------------------------|----------|-----------------------------|
| Branch Less Than             | blt      | if(R[rs]<R[rt]) PC = Label  |
| Branch Greater Than          | bgt      | if(R[rs]>R[rt]) PC = Label  |
| Branch Less Than or Equal    | ble      | if(R[rs]<=R[rt]) PC = Label |
| Branch Greater Than or Equal | bge      | if(R[rs]>=R[rt]) PC = Label |
| Load Immediate               | li       | R[rd] = immediate           |
| Move                         | move     | R[rd] = R[rs]               |

**NOTE THE FOLLOWING (also) on the FRONT PAGE:**

1. List of **Pseudoinstructions**

- *Remember: In CS64, you are ONLY allowed to use these in addition to the pseudoinstruction **la** (load memory address)*
- See slides from previous lecture!

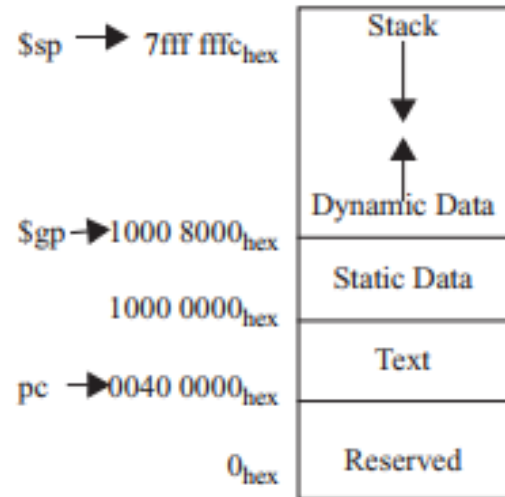
**REGISTER NAME, NUMBER, USE, CALL CONVENTION**

| NAME      | NUMBER | USE                                                   | PRESERVED ACROSS A CALL? |
|-----------|--------|-------------------------------------------------------|--------------------------|
| \$zero    | 0      | The Constant Value 0                                  | N.A.                     |
| \$at      | 1      | Assembler Temporary                                   | No                       |
| \$v0-\$v1 | 2-3    | Values for Function Results and Expression Evaluation | No                       |
| \$a0-\$a3 | 4-7    | Arguments                                             | No                       |
| \$t0-\$t7 | 8-15   | Temporaries                                           | No                       |
| \$s0-\$s7 | 16-23  | Saved Temporaries                                     | Yes                      |
| \$t8-\$t9 | 24-25  | Temporaries                                           | No                       |
| \$k0-\$k1 | 26-27  | Reserved for OS Kernel                                | No                       |
| \$gp      | 28     | Global Pointer                                        | Yes                      |
| \$sp      | 29     | Stack Pointer                                         | Yes                      |
| \$fp      | 30     | Frame Pointer                                         | Yes                      |
| \$ra      | 31     | Return Address                                        | No                       |

2. List of all 32 **Registers**

- a) and their **ID numbers**,
- b) and their **typical uses**
- c) and their **calling convention**  
*(more about this part in later lectures)*

## MEMORY ALLOCATION



## DATA ALIGNMENT

| Double Word |      |          |      |          |      |          |      |
|-------------|------|----------|------|----------|------|----------|------|
| Word        |      |          |      | Word     |      |          |      |
| Halfword    |      | Halfword |      | Halfword |      | Halfword |      |
| Byte        | Byte | Byte     | Byte | Byte     | Byte | Byte     | Byte |
| 0           | 1    | 2        | 3    | 4        | 5    | 6        | 7    |

## SIZE PREFIXES (10<sup>x</sup> for Disk, Communication; 2<sup>x</sup> for Memory)

| SIZE                               | PRE-FIX | SIZE                               | PRE-FIX | SIZE              | PRE-FIX | SIZE              | PRE-FIX |
|------------------------------------|---------|------------------------------------|---------|-------------------|---------|-------------------|---------|
| 10 <sup>3</sup> , 2 <sup>10</sup>  | Kilo-   | 10 <sup>15</sup> , 2 <sup>50</sup> | Peta-   | 10 <sup>-3</sup>  | milli-  | 10 <sup>-15</sup> | femto-  |
| 10 <sup>6</sup> , 2 <sup>20</sup>  | Mega-   | 10 <sup>18</sup> , 2 <sup>60</sup> | Exa-    | 10 <sup>-6</sup>  | micro-  | 10 <sup>-18</sup> | atto-   |
| 10 <sup>9</sup> , 2 <sup>30</sup>  | Giga-   | 10 <sup>21</sup> , 2 <sup>70</sup> | Zetta-  | 10 <sup>-9</sup>  | nano-   | 10 <sup>-21</sup> | zepto-  |
| 10 <sup>12</sup> , 2 <sup>40</sup> | Tera-   | 10 <sup>24</sup> , 2 <sup>80</sup> | Yotta-  | 10 <sup>-12</sup> | pico-   | 10 <sup>-24</sup> | yocto-  |

The symbol for each prefix is just its first letter, except μ is used for micro.

## NOTE THE FOLLOWING on the BACK PAGE:

### 1. How **Memory is Allocated** in MIPS

Note the default address values for each of the following:

- the stack,
- the heap (“dynamic data”),
- static data,
- text (program) data

### 2. The (interesting point that) stack **grows** from high addresses to low addresses (much more on this next week)

### 3. How bytes **align** with **words**

### 4. The basic **size prefixes**

# How Does Multiplication Work??

1

*Recall:*

If you're multiplying by a power of 2, it's always easiest to use **shift-left-logical (sll)**

Example:

```
li $t1, 4
```

```
sll $t0, $t1, 4           # What value will be in reg. $t0?
```

```
# Answer: Value in $t0 will be 64
```

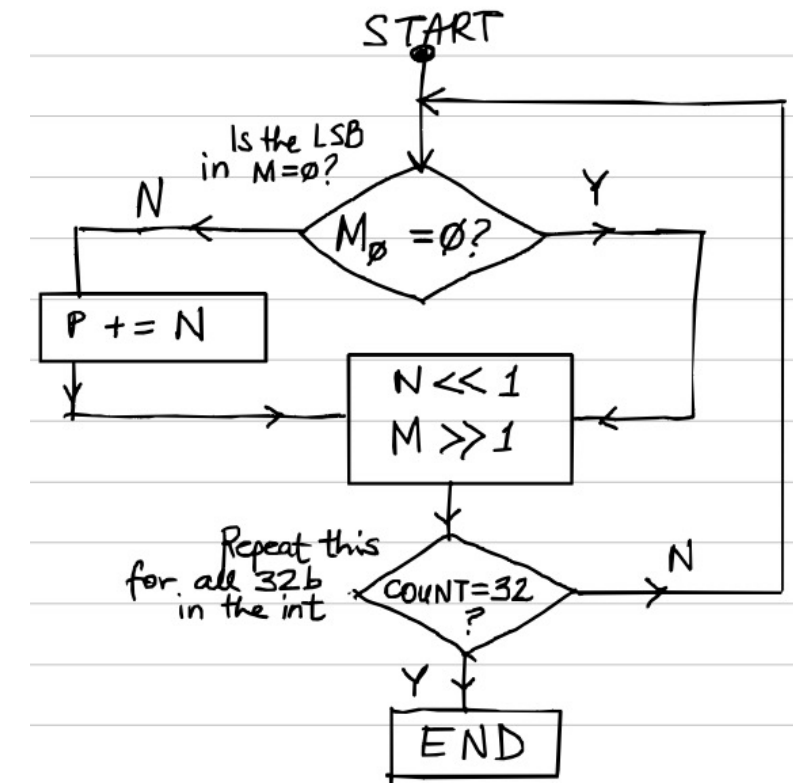
```
# Why?   : Because  $\$t0 = 4 \times 2^4 = 4 \times 16 = 64$ 
```

# How Does Multiplication Work??

2

**FOR GENERAL MULTIPLICATION**, however, the ALU uses an internal technique with the hardware that does something like the following:

- Abstract: Start with 4 x 32-bit data containers:
  - Product result (**P**), which is initialized to 0
    - **P** actually takes up 2 x 32-bit containers
  - *Multiplier* (**M**) and *Multiplicand* (**N**)
- Follow the algorithm shown here::::::::::::::::::::::::::::::::::::::::::
  - Simplified to show positive number multiplication only...
- Example in 4-bits (instead of 32 bits):
  - $M = 0101$  (i.e., 5)
  - $N = 0011$  (i.e., 3)
  - Expect **P** to end up as **1111** (i.e., 15)



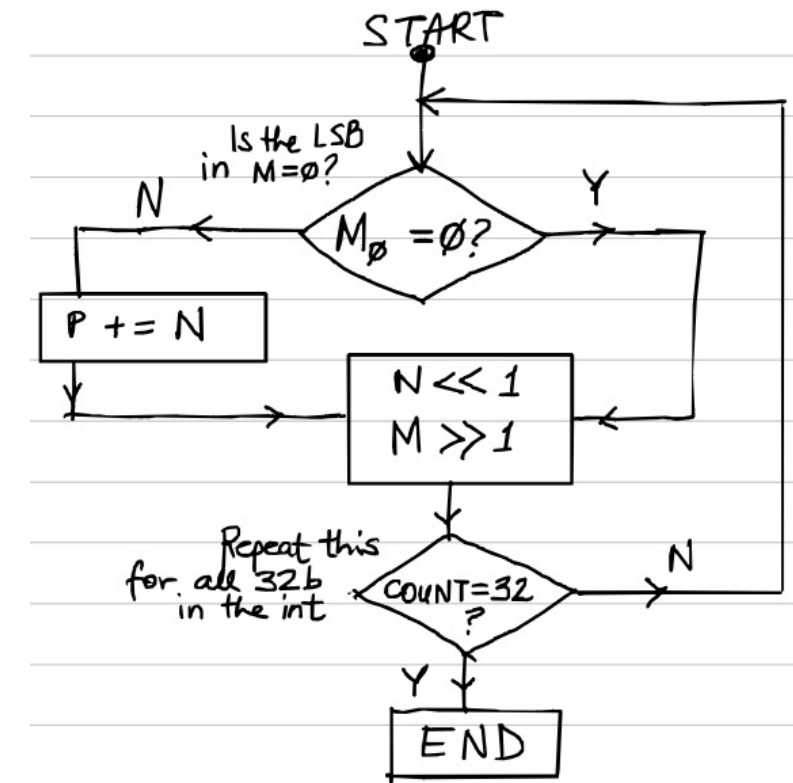
# How Does Multiplication Work??

## EXAMPLE

- Example in 4-bits (instead of 32 bits):
  - M = **0101** (i.e., 5) and N = **0011** (i.e., 3)

- P = 0
- $M_0 = 1 \Rightarrow P += N = 0011$
- $N \ll 1 \Rightarrow N = 0110$
- $M \gg 1 \Rightarrow M = 0010$
- Repeat? (Yes)
- $M_0 = 0 \Rightarrow$  Don't change P
- $N \ll 1 \Rightarrow N = 1100$
- $M \gg 1 \Rightarrow M = 0001$
- Repeat? (Yes)
- $M_0 = 1 \Rightarrow P += N = 0011 + 1100 = \mathbf{1111}$

*It continues a little more from here, but every bit is zero, so the answer doesn't change...*



# How Does Multiplication Work??

3

To do General Multiplication in MIPS, use a “**co-processor instruction**” called **mult**  
BUT you have to use it in conjunction with another “co-processor instruction”: **mflo**

Example:

```
mult $t1, $t2  
mflo $t0
```

This multiplies the 32b value in **\$t1** by the 32b value in **\$t2** and places the 32b result in **\$t0**

*Question: How many bits do you think we might need for the answer of a 32b x 32b?!*

*Answer: UP TO 64 b !!!*

- In CS64, we can always safely assume that any multiplication results we do *will still fit in 32 bits*
  - In general, you cannot count on this, because the answer could be up to 64 bits long

*Cannot do this using **sll**  
(which is always a preference b/c it takes only 1 cycle,  
whereas **mult** can take **several cycles** to conclude)*

# mult Instruction Example: $t2 = 5 * -7$

To multiply 2 ints together, remember the following rules (as seen with this example):

```
li $t0, 5
li $t1, -7
mult $t1, $t0
mflo $t2
```

*Essentially, this is: **do 5 x -7, put result (-35) in \$t2**  
Noting that these are each 32-bit integers*

**mult** cannot be used with an 'immediate' value (e.g., you **cannot** do **mult 5, -7** or **mult \$t0, -7**)

First, we load our *multiplier* into a register (\$t0)

Then we multiply this with our *multiplicand* (\$t1)

And we finally put the (lower 32-bit) result in the final register (\$t2) using the **mflo** instruction

NOTE: There's a (**banned**) pseudoinstruction called **mul** --- **you are not allowed to use it!!** (so don't confuse it with **mult**)

# iClicker!

*What will this program do when you run it on spim?*

- A. It will print the number **1**
- B. It will print the number **10**
- ☒ C. It will print the number **1010**
- D. It will run but not print out anything
- E. It will not run because of an error

```
.text
main:
    li $t0, 1
    li $t1, 9
    add $a0, $t0, $t1
    move $v0, $t0
    syscall
    syscall
    move $v0, $a0
    syscall
```

# iClicker!!

*What will this program do when you run it on spim?*

- A. It will print the number **0**
- ☒ B. It will print the number **15**
- C. It will print the number **-1**
- D. It will print the number **21**
- E. It will not run because of an error

```
.text
main:
    li $t0, 13
    li $t1, 9
    and $t0, $t0, $t1
    ori $a0, $t0, 7
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

# A Note About Operands

Operands in arithmetic instructions are limited and are done in a certain order

- Direct arithmetic operations on integers are always done in the **ALU**
- ALU in/out are connected to the **registers (and sometimes decoder) ONLY, i.e., not to mem.**

Examples:  $f = (g + h) - (i + j)$

$f = g * h - i$

- The order is prescribed by the parentheses or by PEMDAS rule
- Let's say, **f, g, h, i, j** are going to be assigned to registers **\$s0, \$s1, \$s2, \$s3, \$s4** respectively
- What would the MIPS assembly code look like in either case?

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```

```
mult $s1, $s2
mflo $t0
sub $s0, $t0, $s3
```

# Conditionals

---

What if we wanted to do: `if (x == 0) { cout << "x is zero"; }`

What do we need to implement this?

- A way to *compare* numbers and assess a Boolean result (i.e. T/F)
- A way to *conditionally execute* code

# Relevant Core Instructions in MIPS

*for use with branching conditionals*

---

Comparing numbers: **set-less-than (slt)**

- **Set** some register (i.e. *make it “1”*)  
if some **\$reg1** less-than some other **\$reg2** is **true**

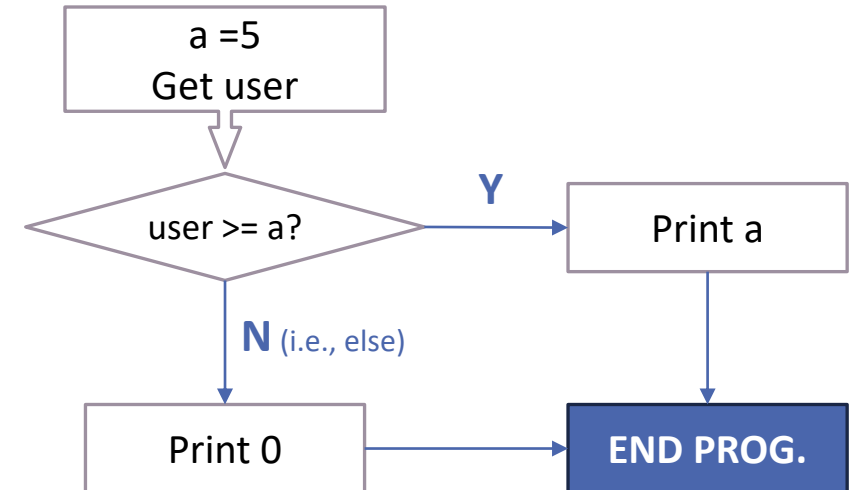
Conditional execution: **branch-on-equal (beq)**  
**branch-on-not-equal (bne)**

- “Go to” some other place in the code (i.e. jump)

# if/else Example

Consider this C++ program:

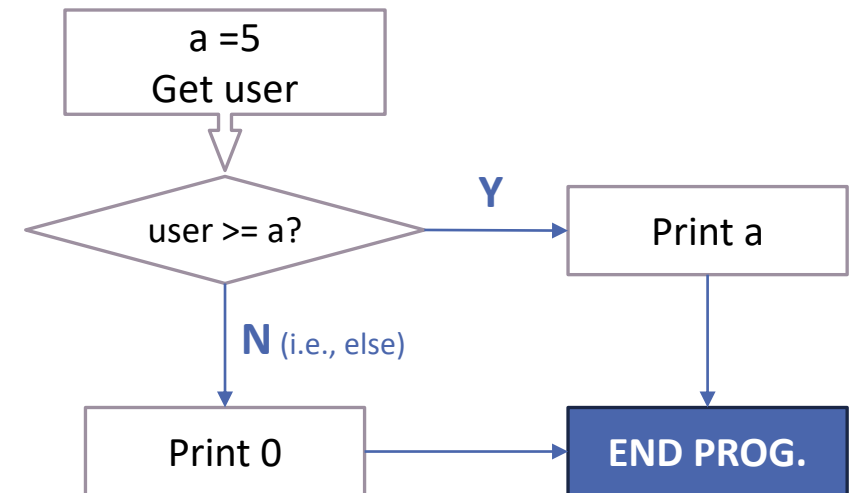
```
int main() {  
    int a = 5, user;  
    cin >> user;  
    if (user >= a) { cout << a; }  
    else { cout << 0; }  
    return 0;  
}
```



*set-less-than* makes the destination register value either 1 or 0, which is super-convenient!

# if/else Example

```
# Make $a0 = 5
# Get user input (ends up in $v0)
# (using slt) set $t0 = 1, if v0 < a0 ; $t0 = 0 if not
# (using bne) if $t0 ≠ 0 (that is, t0=1) then go to ELSE:
# OTHERWISE:
#     print a0
#     jump to the end of program
# ELSE: (that is, if v0 < a0)
#     print 0
#     jump to the end of program (necessary?)
# END OF PROGRAM
```



# DEMO !!!

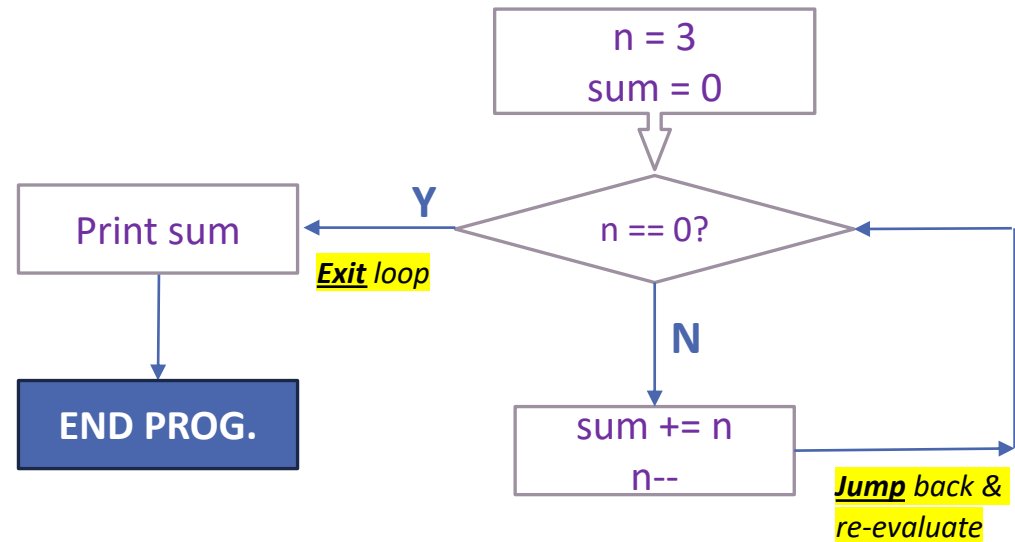
Demo Code File: **ifelse\_ex1.asm**

# A Loop Example!

*Where's the branching here???*

Consider this C++ program:

```
int main() {  
    int n = 3, sum = 0;  
    while (n != 0) {  
        sum += n;  
        n--;  
    }  
    cout << sum;  
    return 0;  
}
```



**DEMO !!!**

Demo Code File: **loop\_ex1.asm**

# To Dos

---



Finish the reading

Work on your new assignment (DUE MONDAY!!)

Come to office hours!! 😊

**Lab on Friday!**

</LECTURE>